**Type class**
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-630004.1
Method **overloading**, like **extension classes** in C#.
Types implement a class by defining a type class **instance**.
i.e. a way of adding **methods** on a type without modifying the type.

Typeclass

```
---- Basics
--
class MyClass c where
  fn :: c -> c

instance MyClass Int where
 fn x = 2 * x
fn (2::Int)        -- 4


class MyClass a b c where         -- ERROR, unless GHC
multiparam extension is enabled
class MyClass where               -- ERROR


-- type variable kind:
-- i.e. whether type variable is monomorphic::*,
unary::* -> *, binary, etc
--
-- monomorphic
class MyClass c where
class MyClass (c::Type) where             -- same with
GHC Kind signature extension
  fn :: c -> Bool
  fn2 :: c a -> a                         -- ERROR.
monomorphic c::* in fn but unary c::* -> * in fn2

-- instances
instance MyClass Int where          -- OK. monomorphic
Int::*
instance MyClass Maybe where        -- ERROR. unary
Maybe a::* -> *
instance MyClass (Maybe a) where    -- OK
instance MyClass (Maybe Int) where  -- OK


-- unary c::*->*
class MyClass c where
class MyClass (c::Type -> Type) where     -- same with
GHC Kind signature extension
  fn :: c a -> a
  fn2 :: c -> Bool                        -- ERROR.
unary c::* -> * in fn but nullary c::* in fn2

-- instances
```

```
instance MyClass Int where           -- ERROR.
monomorphic Int::*
instance MyClass Maybe where         -- OK
instance MyClass Either where        -- ERROR. binary
Either::* -> * -> *
instance MyClass (Either a) where    -- OK
instance MyClass (Either Int) where  -- OK


---- context aka type class specification
--
-- Other classes assumed by this class
-- i.e. in OOP terms, super classes
--
class (Functor c, Traversable c) => MyClass c where
 -- MyClass assumes Functor and Traversable. i.e. super
classes
 ...
class (Functor c, Eq c) => MyClass c where
 -- ERROR. Conflicting kinds. Functor (c::Type->Type)
but Eq (c::Type)

-- e.g. often used to define hierarchy
class Num c where
class Num c => Real c where
class Num c => Fractional c where
class (Real c, Enum c) => Integral c where
class (Real c, Fractional c) => RealFrac c where
class Fractional c => Floating c where
class (RealFrac c, Floating c) => RealFloat c where



---- declarations
--
-- i.e. methods
class MyClass c where
 myfn :: c -> c -> Bool        -- function
 myconst :: c                  -- may be monomorphic
 (#+) :: c -> c -> Bool        -- operator
 myfn x y = False              -- default

 label::String                 -- ERROR. Must contain
type variable somewhere
 doSomething::Int -> Int

-- case: empty declaration
class MyClass c where                        -- OK
class (Read c, Show c) => MyClass c where    -- Used to
combine super classes


---- fixity
--
-- Only for operators
```

```
class MyClass c where
 (#+) :: c -> c -> c
 infixl 9 #+            -- left
 infixr 9 #+            -- right
 infix 9 #+             -- none

 fn :: c -> c
 infix 9 f              -- ERROR. Not an operator

 (#+) :: c -> c -> c
 (#++) :: c -> c -> c
 infixl 9 #+, #++       -- multiple

-- can be defined anywhere, including outside + before
class
 infixl 9 #+, #++             -- OK. Outside + before
class
class MyClass c where
 infixl 9 #+, #++             -- OK. Inside + top
 (#+) :: c -> c -> c
 (#++) :: c -> c -> c
 infixl 9 #+, #++             -- OK. Inside + bottom
infixl 9 #+, #++              -- OK. Outside + after
class
```

Type class instance

**Instance rules**
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/instances.html#

**Patterson condition**
Defined in Haskell 98.
A rule about how **type variables** can be used in class instances.
In Haskell 98, variables must be used exactly as in the original **type definition**.
i.e.
**type variables** in class instance if and only if type variables of a **polymorphic type**.

e.g. monomorphic class parameter (a :: Type)

```
class MyClass (a :: *) where ...                -- Monomorphic parameter a

-- Monomorphic type instance
instance MyClass Int where                      -- OK
instance MyClass a where                        -- ERROR
Monomorphic instance cannot be a variable
instance Show a => MyClass a where              -- ERROR

-- Polymorphic type instance
instance MyClass (Either a b) where             -- OK
instance (Show a, Show b)                       -- OK
  => MyClass (Either a b) where
```

```
instance MyClass (Either a a) where            -- ERROR
variables need to be distinct
instance MyClass (Either Int a) where          -- ERROR
cannot use non-variable
instance MyClass (Either Int String) where     -- ERROR
```

e.g. higher-kinded class with polymorphic parameter (m :: Type -> Type)

```
class MyClass (m :: * -> *) where ...          --
polymorphic parameter m

instance MyClass Maybe where                   -- OK
instance MyClass (Either a) where              -- OK
instance Show a => MyClass (Either a) where    -- OK
instance MyClass (Either Int) where            -- ERROR
Must be a type variable
```

One reason for this restriction is ensure that instances do not **conflict**. i.e. a type belongs to **exactly one instance**.
e.g. if a **monomorphic parameter** can be a variable

```
instance MyClass Int where ...
instance MyClass a where ...                   --
conflict for Int
```

e.g. if **polymorphic type** can have a concrete type

```
instance MyClass (Either a Int) where ...
instance MyClass (Either String a) where ...   --
conflict for (Either String Int)
```

```
---- Basic
--

class MyClass (c::Type) where
  isValid :: c -> Bool

instance MyClass Char where
  isValid = isAlphaNum
--
instance MyClass Int where ...
instance MyClass [Int] where ...
instance MyClass [a] where ...          -- OK with
variable
instance MyClass [] where ...           -- ERROR. kind
mismatch. c::* but []::*->*
instance MyClass (Maybe a) where ...
instance MyClass Maybe a where ...      -- ERROR. Must
have ()
```

```
---- Higher-kinded classes
--

class MyMonomorphicClass (c::Type) where
 fn :: c -> Bool
class MyUnaryClass (c::Type->Type) where
 fn2 :: c a -> Bool
class MyBinaryClass (c::Type->Type->Type) where
 fn3 :: c a b -> Bool

-- Monomorphic class
-- i.e. type ::*
instance MyMonomorphicClass Int where
instance MyMonomorphicClass [Int]
where ...             -- ERROR
instance MyMonomorphicClass [a]
where ...             -- OK with variable
instance MyMonomorphicClass []
where ...                 -- ERROR
instance MyMonomorphicClass (Either Int Int)
where ...    -- ERROR
instance MyMonomorphicClass (Either a b)
where ...        -- OK With variable
instance MyMonomorphicClass (Either Int)
where ...        -- ERROR. unary Either Int::* -> *
instance MyMonomorphicClass (Either a)
where ...        -- ERROR
instance MyMonomorphicClass Either
where ...             -- ERROR  binary Either::* -> * ->
*

-- unary type
-- i.e. type :: * -> *
instance MyUnaryClass Int where                    --
ERROR. monomorphic Int::*
instance MyUnaryClass [] where                     -- OK
instance MyUnaryClass [Int] where                  --
ERROR
instance MyUnaryClass Either where ...             --
ERROR. binary Either:: * -> * -> *
instance MyUnaryClass (Either a) where ...         -- OK
instance MyUnaryClass (Either a b) where ...       --
ERROR. monomorphic Either a b::*


---- special syntax: list, tuple, function
--
-- list
instance MyUnaryClass [] where ...
instance MyMonomorphicClass ([] Int) where ...
instance MyMonomorphicClass [Int] where ...         --
OK

-- tuple
```

```
instance MyBinaryClass (,) where
instance MyUnaryClass ((,) a) where
instance MyMonomorphicClass ((,) a b) where
instance MyUnaryClass (a,) where          -- ERROR.
Cannot use usual tuple notation
instance MyMonomorphicClass (a,b) where       -- ERROR

-- function
instance MyBinaryClass (->) where ...
instance MyUnaryClass ((->) a) where ...
instance MyMonomorphicClass ((->) a b) where ...
instance MyUnaryClass (a ->) where ...         -- ERROR.
Cannot use usual function notation
instance MyMonomorphicClass (a -> b) where ...     --
ERROR.



----  With context
--
instance Read a => MyClass [a] where
instance (Read a, Show a) => MyClass [a] where       --
multiple

-- self in context: i.e. class itself may appear in
context
instance MyClass a => MyClass (Identity a) where     --
OK. MyClass is both in the instance and the context
```

```
One common usage is classes on wrapper types
i.e. classes which inherit inner value

-- e.g. Ref: https://downloads.haskell.org/ghc/latest/
docs/libraries/base-4.17.0.0/Prelude.html
instance Eq a => Eq [a]
instance (Eq a, Eq b) => Eq (a,b)
instance (Eq a, Eq b, Eq c) => Eq (a,b,c)
instance (Eq a, Eq b, Eq c, Eq d) => Eq (a,b,c,d)
instance Functor f => Functor(Kleisli f a)
instance (Functor f, Functor g) => Functor
(Data.Functor.Product f g)
instance (Functor f, Functor g) => Functor
(Data.Functor.Sum f g)
instance (Functor f, Functor g) => Functor
(Data.Functor.Compose f g)
```

```
-- rule: instance of type only
-- i.e. no class instance of a class
instance MyClassA a => MyClassB a where        -- ERROR.
Instance must be a type, not class

                                               -- See
instance rules above.
```

```
-- rule: unique type signature
-- i.e. no same signature even with different context
instance (Num a) => MyClass [a] where          -- OK.
First instance of MyClass [a]
instance (Show a) => MyClass [a] where         -- ERROR.
Second instance of MyClass [a].
                                               -- See
instance rules above.
```

==========

Derivation

Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-680004.2

**Derived class instances**
**Derive** classes for a new type from classes of the underlying types.
In many simple cases, can just apply **term-wise**.
e.g.

```
data A a b = A a b deriving (Show, Eq)

-- same as
instance (Eq a, Eq b) => Eq (A a b) where
  (A a b) == (A c d) = (a == b) && (c == d)
```

Derivable classes are **fixed**
- Eq, Ord: values are compared **term wise**
Ordering is **lexicographic** i.e. from the left.
e.g.

```
data Pair a b = Pair a b deriving (Eq,Ord)

-- Eq
Pair 1 1 == Pair 1 1        -- True
Pair 1 2 == Pair 1 1        -- False
Pair 1 (*) == Pair 1 (*)    -- ERROR. (Int -> Int) is
not an Eq

-- Ord: lexicographic
Pair 1 1 < Pair 1 2         -- True
Pair 1 1 < Pair 10 0        -- True
Pair 1 1 < Pair 0 10        -- False
Pair 1 (*) < Pair 1 (*)     -- ERROR
```

- **Enum**: only for **monomorphic sums**
Just enumerates the constructors

```
data MyValues = A | B | C | D | E deriving (Enum)
[A..]           -- [A,B,C,D,E]
[A,C..]         -- [A,C,E]
```

```
data MyValue a = MyValue a deriving Enum            --
ERROR Not supported
```

- **Bounded**:
Two subcases
a) type derives **enum**: i.e. only monomorphic sums
The bounds are the first and last constructors.
e.g.

```
data MyValues = A | B | C | D | E deriving
(Enum,Bounded)
minBound::MyValues        -- A
maxBound::MyValues        -- E
```

b) exactly one data constructor: just apply termwise

```
data MyPair a b = MyPair a b deriving (Bounded)

minBound::MyPair Int Char                -- MyPair
(-9223372036854775808) '\NUL'
MyPair (minBound::Int) (minBound::Char)  -- same
maxBound::MyPair Int Char                -- MyPair
9223372036854775807 '\1114111'
MyPair (maxBound::Int) (maxBound::Char)
minBound::MyPair Int Double              -- ERROR Double
is not a Bounded
```

- **Show, Read**:

```
----  Basics
--
data A a = A a deriving Show
data A a = A a deriving (Show, Read)        -- multiple
-- supported
data A a = A a deriving (Show, Read)        -- deriving
in new algebraic type
type A a = [a] deriving (Show, Read)        -- ERROR
newtype A a = A a deriving (Show, Read)     -- deriving
in new type
-- supported derivable classes
data A a = A a deriving (
 Eq,
 Ord,
 Enum,
 Bounded,
 Show,
 Read,
)



---- GHC extension: more derivable classes
```

```
-- Ref: https://downloads.haskell.org/ghc/latest/docs/
users_guide/exts/control.html
{-# LANGUAGE
DeriveFunctor, DeriveFoldable,DeriveTraversable #-}
{-# LANGUAGE DeriveLift,DeriveGeneric,DeriveDataTypeable
#-}
data A a = A a deriving (
 Typeable,
 Generic,
 Lift,
 Functor,
 Foldable,
 Traversable,
)
```

==========

GHC extension

GHC extension: Default signatures
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/default_signatures.html

```
-- Basic syntax
class DoSomething a where
  doSomething :: a -> a
  default doSomething :: Num a => a -> a
  doSomething = negate
```

GHC extension: FlexibleInstances, UndecidableInstances
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/instances.html

**FlexibleInstances**
By default, GHC has various restrictions on type instances.
Flexible instances relaxes these

```
-- Without FlexibleInstances
class DoSomething a where
  doSomething :: a -> a
instance DoSomething Int where                    -- OK
instance DoSomething a where                      --
ERROR. unary type cannot be a type variable
instance DoSomething (Maybe a) where              -- OK
instance DoSomething (Maybe Int) where            --
ERROR. polymorphic type must have same type variables as
the type definition
instance DoSomething (Either a a) where           --
ERROR
instance Show a => DoSomething a where            --
ERROR
instance Show a => DoSomething (Maybe a) where    -- OK
```

```
-- With FlexibleInstances
{-# LANGUAGE FlexibleInstances #-}
class DoSomething a where
  doSomething :: a -> a
instance DoSomething Int where                      -- OK
instance DoSomething a where                        -- OK
instance DoSomething (Maybe a) where                -- OK
instance DoSomething (Maybe Int) where              -- OK
instance DoSomething (Either a a) where             -- OK
instance Show a => DoSomething a where              -- OK
instance Show a => DoSomething (Maybe a) where      -- OK
```

GHC extension: Overlap
Ref: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/instances.html#

**Overlap**
By default, GHC does not allow class instances to **overlap**. i.e. a type cannot belong to more than **one instance** of the same class.
The overlap extension relaxes this:

| | |
|---|---|
| instance {-# **OVERLAPPABLE** #-} MyClass a | **allow override** by a more specific instance |
| instance {-# **OVERLAPPING** #-} MyClass Int | **force override** a less specific instance |
| instance {-# **OVERLAP** #-} MyClass a | both |

**usage**: default class instances + specific instances

```
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE UndecidableInstances #-}

class MyShow a where
  myShow :: a -> String

instance Show a => MyShow a where                    --
default
  myShow = show
instance {-# OVERLAPPING #-} MyShow Int where        --
Int specific
  myShow v = "INT " ++ show v
instance {-# OVERLAPPING #-} MyShow Bool where       --
Bool specific
  myShow v = "BOOL " ++ show v

-- e.g.
myShow "A"           -- "A"
myShow 1.1           -- "1.1"
myShow (1::Int)      -- "INT 1"
myShow True          -- "BOOL True"
myShow ()            -- "()"
```

e.g. similar with default signature

```
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE UndecidableInstances #-}
{-# LANGUAGE DefaultSignatures #-}
{-# LANGUAGE StandaloneDeriving #-}
{-# LANGUAGE DeriveAnyClass #-}

class MyShow2 a where
  myShow2 :: a -> String
  default myShow2 :: Show a => a -> String
  myShow2 = show

deriving instance Show a => MyShow2 String           --
default
instance  {-# OVERLAPPING #-}  MyShow2 Int where     --
Int specific
  myShow2 v = "INT " ++ show v
instance  {-# OVERLAPPING #-}  MyShow2 Bool where    --
Bool specific
  myShow2 v = "BOOL " ++ show v
```

```
--
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE UndecidableInstances #-}
class MyClass a where

-- With force override
instance Show a => MyClass a
where                           -- base
instance MyClass Int
where                               -- ERROR. Cannot
override
instance {-# OVERLAPPING #-} MyClass Int
where             -- OK. Override
instance {-# OVERLAP #-} MyClass Int
where                  -- OK. Override

-- With overridable base instance
instance {-# OVERLAPPABLE #-} Show a => MyClass a
where        -- OK. base can be overridden
instance {-# OVERLAP #-}  Show a => MyClass a where
    -- OK. Same
instance MyClass Int where
    -- OK.
```

GHC extension: multiple parameter class + functional depedencies
Multiple parameters: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/multi_param_type_classes.html
Functional dependencies: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/functional_dependencies.html

**Multiple parameter class**
A class defined on a **combination of types** i.e. sum, instead of a single type.
Needed when we want to have **different implementations** of the **same method** depending on a **combination** of the types involved
e.g. Num class (+) :: a -> a -> a

| | |
|---|---|
| (1 :: Int) + (1 :: Int) | OK. Implemented by Num Int |
| (1 :: Double) + (1 :: Double) | OK. Implemented by Num Double |
| (1 :: Int) + (1 :: Double) | ERROR. |

e.g.

```
class MyClass a b where
  (#+) :: a -> b -> a
-- Int + Int
instance MyClass Int Int where
  (#+) x y = x + y
- Int + Double
instance MyClass Int Double where
  (#+) x y = x * round y
- value + function
instance MyClass a (a -> a) where
  (#+) x f = f x

x = (1::Int) #+ (1::Int)               -- 2=1+1
s = (1::Int) #+ (1::Double)            - 1=1*1
y = (1::Int) #+ ((*10)::Int -> Int)    - 10=1*10
```

Impossible without multiple parameters

```
class MyClass a where
  (#+) :: a -> b -> a
instance MyClass Int where
 combine x y = x + y            -- NO. Can't do anything
with y since we don't know anything about it
 combine x y z = x              -- OK.
```

**Functional dependency**
A subset of parameters **determines** the other parameters.

| | |
|---|---|
| MyClass a b c d \| a b -> c | if a b then c |
| instance MyClass () () Int () | if a=() b=() then c=Int |
| instance MyClass () () Int Bool | OK. d is free. |
| instance MyClass () () Char () | ERROR. c must be Int |
| instance MyClass () Int () () | OK. if a=() b=Int then c=() |

Needed to prevent **class ambiguities** i.e. an expression matches **multiple** class instances
e.g.

```
-- without dependencies
class MyClass a b where
 myValue :: a            -- BAD. Ambiguous.
```

```
 myFn :: a -> b

-- e.g.
myValue @Int             -- myValue of MyClass Int ?


-- with dependencies
class MyClass a b | a -> b where
 myValue :: a
 myFn :: a -> b

instance MyClass () () where ...
instance MyClass Int Char where ...
instance MyClass () Int where ...         -- ERROR

-- e.g.
myValue @()               -- myValue of MyClass () ()
myValue @Int              -- myValue of MyClass Int
Char
```

```
---- Multiple parameter class
--
-- Ref: https://downloads.haskell.org/ghc/latest/docs/
users_guide/exts/multi_param_type_classes.html
class MyClass a b c where
 combine :: a -> b -> c -> a


---- Functional dependencies
--
class MyClass a b | a -> b
class MyClass a b c | a -> b, b -> c
class MyClass a b c d | a b -> c, a b -> d
class MyClass a b c d | a b -> c d, a c -> d

-- e.g.
class MyClass a b | a -> b where
instance MyClass Int Int where        -- No other
MyClass Int * allowed
instance MyClass Double Int where     -- OK
instance MyClass Int Double where     -- ERROR.

-- e.g. multiple dependents
class MyClass a b c | a b -> c where
instance MyClass Int Int Int where       -- No
other MyClass Int Int * allowed
instance MyClass Int Double Int where    -- OK
instance MyClass Int Int Double where    -- ERROR.

-- e.g. multiple dependencies:
-- All dependencies must be met.
-- i.e. fails if at least one fails.
class MyClass a b c d | a -> b, c -> d where  -- fails
```

```
if a->b OR c->d fails
class MyClass a b c d | a c -> b where        --
Actually same
class MyClass a b c d | a c -> d where        -- same
instance MyClass Int Int Int Int where           -- No
other MyClass Int * * * OR MyClass * * Int * allowed
instance MyClass Double Int Int Int where        -- OK
instance MyClass Int Int Double Int where        -- OK
instance MyClass Int Double Int Int where        --
ERROR.
instance MyClass Int Int Int Double where        --
ERROR.
instance MyClass Int Double Double Double where  --
ERROR. Conflicts with MyClass Int * * *
instance MyClass Double Double Int Double where  --
ERROR. Conflicts with MyClass * * Int *
```

GHC extension: derive Functor, Foldable, Traversable
Functor: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/deriving_extra.html#extension-DeriveFunctor
Foldable: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/deriving_extra.html#extension-DeriveFoldable
Traversable: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/deriving_extra.html#extension-DeriveTraversable

**Basic idea**
Just applies the appropriate operation **term-wise**

```
e.g.
import Data.Monoid
data A b a = A a [a] b deriving (Functor, Foldable,
Traversable, Show)

(*2) <$> A 1 [1,2] True           -- map. A 2 [2,4]
True
foldMap Sum $ A 1 [1,2] True      -- fold. Sum {getSum =
4}
traverse print $ A 1 [1,2] True   -- traverse. Prints '1
1 2'
```

**Rules**
- **multi parameter**: only applies on **last parameter**
e.g.

```
-- only applies on last parameter=a
-- b,c are ignored.
data A c b a = A a b c [a] deriving (Functor, Foldable,
Traversable, Show)

(*10) <$> A 1 True "a" 2 [1..3]               -- A 10
True "a" [10,20,30]
(*10) <$> A 1 2 "a" [1..3]                    -- A 10 1
```

```
"a" [10,20,30]
foldMap Data.Monoid.Sum $ A 1 2 "a" [1..3]    -- Sum
{getSum=7}  7=1+1+2+3
```

-- **operation:**
Applied **term-wise** according to the following rule
e.g. if Functor

```
data A m b a = A a b [a] (m a) deriving (Functor,
Foldable, Traversable, Show)
f <$> A r s t u
```

3 possibilities:
a) if parameter is **single**, apply f
b) if parameter is an **inner parameter** of another type, then apply (f<$>)
c) otherwise, apply id
i.e.

```
f <$> A r s t u
 = A
  (f a)       rule a)
  b           rule c)
  f <$> t     rule b)
  f <$> u     rule b)
```

```
-- Basics
--
{-# LANGUAGE DeriveFunctor #-}
{-# LANGUAGE DeriveFoldable #-}
{-# LANGUAGE DeriveTraversable #-}
data A a = A a deriving (Functor)
data A c b a = A a deriving (Functor)          --
derives on last parameter=a only. b,c are ignored.
data A = A deriving (Functor)                  -- ERROR
data A a = A a deriving (Foldable)             -- ERROR.
Need Functor
data A a = A a deriving (Functor, Foldable)    -- OK


-- case: Functor
--
-- Just map each term.
data A m b a = A a b [a] (m a) deriving (Functor, Show)
f <$> A r s t u
```

```
A (
 f r           -- r::a
 s             -- s::b i.e. unapplied
 f <$> t       -- t::[a] i.e. inner param
 f <$> u       -- u::m a i.e. inner param
)
```

```
-- case: Foldable
--
-- Just fold each term.
data A m b a = A a b [a] (m a) deriving
(Functor,  Foldable, Show)
foldr f init <$> A r s t u
```

```
A (
 f r init             -- r::a
 s                    -- s::b i.e. unapplied
 foldr f init t       -- t::[a] i.e. inner param
 foldr f init u       -- u::m a i.e. inner param
)
```

```
-- case: Traversable
--
-- Just traverse each term, but assemble in the
applicative with <*>
-- i.e. result is
--   A x y = (A) x y           functor, foldable
--   pure (A) <*> x <*> y      traversable
--
data A m b a = A a b [a] (m a) deriving
(Functor,  Foldable, Traversable, Show)
traverse f $ A r s t u
```

```
pure (A)            -- constructor of A i.e.
 <*> (f r)            -- r::a
 <*> pure s           -- s::b i.e. unapplied
 <*> traverse f t     -- t::[a] i.e. inner param
 <*> traverse f u     -- u::m a i.e. inner param
```

```
-- GOTCHA: parameter needs to be inner to be mapped
--
-- e.g.
data A a = A a deriving (Functor, Show)
(*2) <$> A [1,2]                                  --
ERROR. Treated as (*2) [1,2], not (*2) <$> [1,2]
(*2) <$> A 1                                      -- A 2
data A a = A [a] deriving (Functor, Show)         -- OK
data A m a = A (m a) deriving (Functor, Show)     -- Also
OK
(*2) <$> A [1,2]                                  -- A
[2,4] as expected


-- e.g.
-- Functor
data A m b a = A a b [a] (m a)
```

```
  deriving (Functor,  Foldable, Traversable, Show)
even <$> A 1 True [1..3] (Just 2)                    --
A False True [False,True,False] (Just True)

-- Foldable
foldMap Sum $ A 1 True [1..3] (Just 2)               --
 Sum {getSum = 9}
foldMap Product $ A 1 True [1..3] (Just 2)           --
Product {getProduct=12}

-- Traversable
justEven x | even x = Just x | otherwise = Nothing
traverse justEven $ A 0 True [0,2] [0,4]    -- Just (A 0
True [0,2] [0,4])
traverse justEven $ A 1 True [0,2] [0,4]    -- Nothing
traverse justEven $ A 0 True [1,2] [0,4]    -- Nothing
traverse justEven $ A 0 True [0,2] [1,4]    -- Nothing
traverse justEven $ A 0 1 [0,2] [0,4]       -- Just (A 0
1 [0,2] [0,4])
```

==========
======

GHC extension - deriving strategies

GHC Extension: DerivingStrategies, StandaloneDeriving
Standalone: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/standalone_deriving.html
Strategies: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/deriving_strategies.html

**DerivingStrategies**
Explicitly specify derivation method
- **stock**: defaults provided by system
  e.g. Show, Eq

- **anyclass** (DeriveAnyclass): classes with default implementation
  e.g. Typeable, Exception

- **newtype (**GeneralizedNewtypeDeriving**)**: newtype **coercion**
  e.g. Monad, MonadReader

- **via** (DerivingVia): generalization of newtype

**StandaloneDeriving**
Separate deriving statement

**Syntax**

```
{-# LANGUAGE DeriveDataTypeable #-}
{-# LANGUAGE DeriveGeneric #-}
{-# LANGUAGE StandaloneDeriving #-}
{-# LANGUAGE DerivingVia #-}
{-# LANGUAGE FlexibleInstances #-}
```

```
import Data.Typeable (Typeable)
import GHC.Generics (Generic)
import GHC.Exception (Exception)
import Control.Monad.Reader
import Control.Monad.Base

-- Basics
--
newtype A m a = A (m a)
  -- Implicit
  deriving (
   -- stock
   Show, Eq,
   -- anyclass
  Typeable,
   -- newtype
  Functor, Applicative, Monad

  )

  -- Explicit with strategies
  deriving stock (Show, Eq)                          --
Show (A m a) if (m a) is Show
  deriving anyclass (Typeable)
  deriving newtype (Functor, Applicative, Monad)   --
newtype
  deriving newtype (MonadReader r, MonadBase b)    --
newtype with unspecified param
                                                   --
i.e. MonadReader r (A m)  if  MonadReader r m
  deriving Monad via m                             --
same with via
  deriving Applicative via m
  deriving Functor via m
  deriving (MonadReader r) via m                   --
unspecified param

-- standalone
deriving stock instance Show (m a) => Show (A m a)
deriving anyclass instance Typeable (A m a)
deriving newtype instance Functor m => Functor (A m)
deriving newtype instance Applicative m => Applicative
(A m)
deriving newtype instance Monad m => Monad (A m)
deriving newtype instance MonadReader r m => MonadReader
r (A m)              -- with unspecified param r
deriving newtype instance MonadBase b m => MonadBase b
(A m)

{-# LANGUAGE KindSignatures #-}
import Data.Kind (Type)
deriving via (m :: Type -> Type) instance Functor m =>
Functor (A m)                 -- same with via
```

```
deriving via m instance Functor m => Functor (A m)
                          -- ERROR "Expected kind * -> *"
deriving via (m :: Type -> Type) instance Applicative m
=> Applicative (A m)
deriving via (m :: Type -> Type) instance Monad m =>
Monad (A m)
deriving via (m :: Type -> Type) instance MonadReader r
m => MonadReader r (A m)  -- with unspecified param r
deriving via (m :: Type -> Type) instance MonadBase b m
=> MonadBase b (A m)
```

GHC extension: DeriveAnyClass
any class: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/derive_any_class.html

**AnyClass**
Derive a class with **default implementation**.

e.g.

```
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DeriveDataTypeable #-}
{-# LANGUAGE DeriveGeneric #-}
{-# LANGUAGE Derivingtrategies #-}

import Data.Typeable (Typeable)
import GHC.Generics (Generic)
import GHC.Exception (Exception)

data MyType = A | B
  deriving stock (Show, Eq)
  deriving anyclass (Typeable, Generic, Exception)
```

**Syntax**

```
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DeriveDataTypeable #-}
{-# LANGUAGE DeriveGeneric #-}

import Data.Typeable (Typeable)
import GHC.Generics (Generic)
import GHC.Exception (Exception)

class MyClass a where
  myId :: a -> a                  -- class with default
implementation
  myId = id

{-# LANGUAGE DerivingStrategies #-}
data A = A
  deriving stock (Eq, Show)
  deriving anyclass (
```

```
    Typeable, Generic, Exception, MyClass,
    Num                                          -- BAD.
Compiles but cannot be used
  )

-- Standalone
{-# LANGUAGE StandaloneDeriving #-}
deriving anyclass instance Typeable A
deriving anyclass instance MyClass A
```

GHC extension: GeneralizedNewTypeDeriving
newtype deriving: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/newtype_deriving.html

**Newtype deriving**
Class derived by **coercion** i.e. ignore wrapping

```
 newtype A = A Int
   deriving stock (Eq, Ord)
   deriving newtype (Num, Real)              Num A   ~  Num Int
                                             Real A   ~  Real Int

 -- with param
 newtype A a = A a
   deriving stock (Eq, Ord)
   deriving newtype (Num, Real)              Num (A a)  ~  Num a, if a is
Num
                                             Real (A a)  ~  Real a, if a is
Real

 -- with higher-kinded classes
 newtype A m a = A (ReaderT String m a)
  deriving (
   Functor, Applicative, Monad,              Functor (A m)                ~
     Functor (ReaderT String m), if m is a functor

     i.e. Functor m
   MonadReader String,                       MonadReader String (A m)     ~
     MonadReader String (ReaderT String m), if m is a monad

     i.e. Monad m
   MonadWriter b,                            MonadWriter b (A m)          ~
     MonadWriter b  (ReaderT String m), if m is a writer

     i.e. MonadWriter b m
   MonadIO                                   MonadIO (A m)                ~
     MonadIO (ReaderT String m) if m is a MonadIO

     i.e. MonadIO m
 )
```

e.g.
Instance via derivation

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
newtype Age a = Age a deriving (Num)
```

Under the hood with coercion

```
{-# LANGUAGE TypeApplications #-}
import Data.Coerce

newtype Age a = Age a

instance forall a. Num a => Num (Age a) where
  abs = coerce @a @(Age a) . abs . coerce
  signum = coerce @a @(Age a) . signum . coerce
  negate = coerce @a @(Age a) . negate . coerce
  fromInteger = coerce @a @(Age a) . fromInteger .
coerce
  x + y = coerce @a @(Age a) (coerce x + coerce y)
  x * y = coerce @a @(Age a) (coerce x + coerce y)
```

**Example**: derive monad stack
e.g.

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE DeriveDataTypeable #-}
{-# LANGUAGE DeriveGeneric #-}

import Control.Monad.Reader
import Control.Monad.Base
import Control.Monad.Trans.Control
import Data.Typeable (Typeable)
import GHC.Generics (Generic)
import Control.Monad.Catch

newtype MyApp a = MyApp {runMyApp :: ReaderT String IO
a}
  deriving stock (Generic, Typeable)
  deriving newtype (
      Functor, Applicative, Monad, MonadIO,
      MonadReader String,
      MonadThrow, MonadCatch, MonadMask,
      MonadBase IO, MonadBaseControl IO
    )
```

**Syntax**

```
-- Basic
--
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GeneralizedNewTypeDeriving #-}

class Show b => MyClass b c

-- case: Monomorphic type
```

```
newtype A = A Int
 deriving newtype (
  Num, Real,                                        --
Num A     ~  Num Int

  -- unspecified param
  MyClass b                                         --
Show b => MyClass b A  ~  MyClass b Int
 )
-- same with standalone
deriving newtype instance Num A
deriving newtype instance Real A
deriving newtype instance Show b => MyClass b A


-- case: Polymorphic type
newtype A a = A a
 deriving newtype (
  Num, Real,                                        --
Num a => Num (A a)  ~  Num a

  -- unspecified param
  MyClass b                                         --
Show b => MyClass b (A a)   ~   MyClass b a
 )
-- same standalone
deriving newtype instance Num a => Num (A a)
deriving newtype instance Real a => Real (A a)
deriving newtype instance Show b => MyClass b (A a)


-- case: higher-kinded classes
--
-- e.g. Functor, Monad
{-# LANGUAGE UndecidableInstances #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE GeneralizedNewTypeDeriving #-}
{-# LANGUAGE KindSignatures #-}
import Control.Monad.Reader
import Control.Monad.State
import Control.Monad.Base
import Control.Monad.TransControl
import GHC.Type (Type)

class SomeClass (w :: Type) (m :: Type -> Type)

newtype MyApp a = MyApp (ReaderT String IO a)
  deriving newtype (
    Functor, Applicative, Monad, MonadIO,           --
Functor MyApp  ~  Functor (ReaderT String IO)
    MonadReader String,                             --
MonadReader String MyApp  ~  MonadReader String (ReaderT
String IO) a
    MonadBase IO,
```

```
    -- unspecified param
    SomeClass (w :: Type)                           --
SomeClass w IO => SomeClass MyApp  ~  SomeClass w
(ReaderT String IO)
  )
-- same standalone
deriving newtype instance Functor MyApp
deriving newtype instance Applicative MyApp
deriving newtype instance Monad MyApp
deriving newtype instance MonadReader String MyApp
deriving newtype instance MonadBase IO MyApp
deriving newtype instance SomeClass w IO => SomeClass w
MyApp
...


-- case: higher-kinded type + classes
newtype MyApp r m a = MyApp (ReaderT r m a)
  deriving newtype (
     Functor, Applicative, Monad,
MonadIO,              -- Functor m => Functor (MyApp r m)
~  Functor (ReaderT r m a)
     MonadReader
r,                                    -- Monad m =>
MonadReader r (MyApp r m)  ~  MonadReader r (ReaderT r m
a)

     -- unspecified param
     MonadWriter
w,                                    -- MonadWriter w m
=> MonadWriter w (MyApp r m)  ~  MonadWriter w (ReaderT
r m a)
     MonadBase (b :: Type ->
Type)                          -- MonadBase b m =>
MonadBase b (MyApp r m)  ~  MonadBase b (ReaderT r m a)
 )
-- same standalone
deriving newtype instance Functor m => Functor (MyApp r
m)
deriving newtype instance Monad m => Applicative (MyApp
r m)
deriving newtype instance Monad m => Monad (MyApp r m)
deriving newtype instance Monad m => MonadReader r
(MyApp r m)
deriving newtype instance MonadWriter w m => MonadWriter
w (MyApp r m)
deriving newtype instance MonadBase b m => MonadBase b
(MyApp r m)
...
```

GHC extension: DerivingVia
via: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/

deriving_via.html

**DerivingVia**
Generalization of **GeneralizedNewtypeDeriving**.
Allows **explicitly** specifying the source instance

| | |
|---|---|
| newtype A = A Int | |
| deriving newtype Num | implicitly use Int |
| deriving Num **via Int** | **explicit** |
| deriving **via Int** instance Num A | standalone |
| deriving newtype instance Num A | |

| | |
|---|---|
| newtype A m a = A m a | |
| deriving newtype Functor | implicitly use m |
| deriving Functor **via m** | **explicit** |
| deriving **via m** | standalone |
| instance Functor m => Functor (A m) | |
| deriving newtype | |
| instance Functor m => Functor (A m) | |

**Example**: implement a specific type of **Semigroup/Monoid**

e.g. Sum vs Product
Applicative vs Alternative

```
{-# LANGUAGE DerivingVia #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

import Control.Applicative
import Data.Monoid

-- e.g. Sum vs Product
newtype A1 = A1 Int
deriving via (Sum Int) instance Semigroup A1
    -- Monoid A1 as Sum
deriving via (Sum Int) instance Monoid A1

newtype A2 = A2 Int
deriving via (Product Int) instance Semigroup A2
    -- Monoid A2 as Product
deriving via (Product Int) instance Monoid A2


-- e.g. Applicative vs Alternative
newtype B1 m a = B1 (m a)
        -- Monoid (B1 m a) with Applicative
  deriving newtype (Functor, Applicative, Monad)
deriving via (Ap m a) instance
  (Applicative m, Semigroup a) => Semigroup (B1 m a)
deriving via (Ap m a) instance
  (Applicative m, Monoid a) => Monoid (B1 m a)

newtype B2 m a = B2 (m a)
  deriving newtype (Functor, Applicative, Monad,
Alternative) -- Monoid (B2 m a) with Alternative
```

```
deriving via (Alt m a) instance
  (Alternative m, Semigroup a) => Semigroup (B2 m a)
deriving via (Alt m a) instance
  (Alternative m, Monoid a) => Monoid (B2 m a)
```

**GOTCHA**: Error "The data constructor X of newtype 'X' is not in scope"
This happens if only the type without the **constructor** is visible

| | |
|---|---|
| import Data.Monoid (Sum) | ERROR. Sum constructor not visible. |
| import Data.Monoid (Sum(..)) | GOOD |

e.g.

```
{-# LANGUAGE DerivingVia #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

import Data.Monoid
(Sum)                                   -- BAD Only type
is imported

newtype A = A Int
deriving via (Sum Int) instance Semigroup
A            -- ERROR "The data constructor ‘Sum’ of
newtype ‘Sum’ is not in scope"
```

Fix

```
import Data.Monoid
(Sum(..))                               -- GOOD
Constructor is imported
import Data.Monoid

deriving via (Sum Int) instance Semigroup
A            -- OK
```

**Syntax**

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE DerivingVia #-}
import GHC.Types (Type)

--case: polymorphic type
newtype A a = A (Maybe a)
  deriving Show via (Maybe a)                       --
Show (A a)  ~  Show (Maybe a)

  -- higher-kinded class
  deriving Functor via Maybe                        --
Functor A  ~  Functor Maybe
  deriving newtype Functor                          --
same
```

```
-- standalone
deriving via (Maybe a) instance Show (A a)

deriving via (Maybe Int) instance Show (A Int)     --
concrete
deriving via Maybe instance Functor A              --
higher-kinded class Functor (f :: Type -> Type)
deriving newtype instance Functor A                --
same


-- case: higher-kinded type
newtype A m a = A m a
  deriving Show via (m a)                      -- Show
(m a) => Show (A m a)  ~  Show (m a)

  -- higher-kinded class
  deriving Functor via m                       --
Functor m => Functor (A m) ~ Functor m
  deriving newtype Functor                     -- same

-- standalone
deriving via (m a)
  instance Show (m a) => Show (A m a)
deriving via (m :: Type -> Type)               --
higher-kinded class Functor (f :: Type -> Type)
  instance Functor m => Functor (A m)
deriving newtype                               -- same
  instance Functor m => Functor (A m)
deriving via Maybe instance Functor (A Maybe)  --
concrete


-- Multiple parameters
class MyClass (m :: Type -> Type) (a :: Type)

instance MyClass Maybe Int

deriving via Int instance MyClass Maybe Double     -- OK
deriving via Maybe instance MyClass IO Int         --
ERROR. Only works for last parameter
```

==========

OLD

This creates restrictions:
- **type** instance only: instance must be a type, not a class
e.g.

```
instance MyClassA a => MyClassB a where        -- ERROR
```

Otherwise, it creates a conflict when a type implements both classes.

```
instance MyClassA a => MyClassB a where
instance MyClassA MyData where
instance MyClassB MyData where                  -- BAD.
Conflicts with `instance MyClassA a => MyClassB a` since
MyClassA MyData is already defined
```

- **unique type signature**: every type signature must be unique
In particular, no signature with **different contexts**.
e.g.

```
instance Num a => MyClass [a] where             -- OK.
First instance of MyClass [a]
instance Show a => MyClass [a] where            -- ERROR.
Second instance of MyClass [a]
```

Forbidden since it creates an overlap if a type implements both classes
But Int implements both Num and Show so

```
[0::Int]         -- ERROR. Num a => MyClass [a] and Show
a => MyClass [a] overlap
```

e.g. derive a type class with **multiple variables**
Can only derive with the **last variable**

```
{-# LANGUAGE DerivingVia #-}
{-# LANGUAGE DataKind #-}

import GHC.TypeLits

class MyClass (a :: Bool) (b :: Symbol) where

instance MyClass 'True "A" where

deriving via "A" instance MyClass 'True "B"           --
OK. MyClass 'True "B" == MyClass 'True "A"
deriving via 'True instance MyClass 'False "A"        --
ERROR "expects an argument of kind 'Symbol', but ''True'
has kind 'Bool'"
                                                      --
i.e. not last variable
```

e.g.

```
{-# LANGUAGE DerivingVia #-}

import Control.Applicative
import Data.Monoid
```

```
newtype MyMaybe a = MyMaybe (Maybe a)
  deriving (Eq, Show) via (Maybe a)
              -- same implementation as Maybe a
  deriving (Functor, Applicative, Monad, Alternative)
via Maybe      -- same implementation  as Maybe monad
  deriving (Semigroup, Monoid) via (Alt Maybe a)
             -- same implementation as Alt Maybe a

-- with standalone deriving
newtype MyMaybe2 a = MyMaybe2 (Maybe a)
deriving via (Maybe a) instance Show a => Show (MyMaybe2
a)
deriving via (Maybe a) instance Eq a => Eq (MyMaybe2 a)
deriving via Maybe instance Functor MyMaybe2
deriving via Maybe instance Applicative MyMaybe2
deriving via Maybe instance Monad MyMaybe2
deriving via (Alt Maybe a) instance Semigroup (MyMaybe2
a)
deriving via (Alt Maybe a) instance Monoid (MyMaybe2 a)
```

**usage [ext: GeneralizedNewTypeDeriving]**: derive newtype by coercion

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

import Fmt

newtype Age a = Age a deriving (
  Show, Eq,                              -- derive
with stock strategy
  Fmt, Num                               -- derive with
newtype strategy
  )

newtype Age a = Age a                    -- with
strategies
  deriving stock (Show, Eq)
  deriving newtype (Fmt, Num)

newtype Age a = Age a                    -- with
strategies + standalone
instance newtype Num a => Num (Age a)
instance newtype Fmt a => Fmt (Age a)

-- e.g.
Age 1 + Age 2                -- Age 3
fmtLn $ build (Age 1)
```

**usage [ext: GeneralizedNewTypeDeriving]**: derive Monad stack in a newtype

**usage [ext: GeneralizedNewTypeDeriving]**: derive Monad stack from a **monad parameter** in a new type
If there is a monad parameter, the derivable classes will depend on the classes of monad parameter

e.g.
newtype MyApp m x = MyApp m x
monad parameter m
deriving instance MonadWriter w m => MonadWriter w (MyApp m)
MyApp is a MonadWriter if m is a MonadWriter

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
{-# LANGUAGE DeriveDataTypeable #-}
{-# LANGUAGE DeriveGeneric #-}

import Control.Monad.Base
import Control.Monad.Catch
import Control.Monad.Except
import Control.Monad.Reader
import Control.Monad.State
import Control.Monad.Trans.Control

newtype MyApp m x = MyApp (ReaderT String m x)
     -- monad parameter m
  deriving newtype
    ( Functor,
      Applicative,
      Monad,
      MonadReader String,
      MonadIO,
      MonadBase b,
      MonadBaseControl b
    )
```

GHC extensions: auto derivations

strategies: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/deriving_strategies.html

4 ways of auto deriving
1) **stock**: usual class derivation of standard derivable classes
i.e.
standard: Eq, Show, Bounded, Enum
GHC: Generics, Typeable, Foldable, Traversable, Functor, Applicative, Monad

2) **any class (extension DeriveAnyClass)**: derive any class which can be used with **default** implementations.
i.e. no minimal set of methods defined

3) **newtype (extension GeneralizedNewtypeDeriving)**: derive a **newtype** type using **coercion**

4) **via (extension DerivingVia, GHC 8.6)**:
See blow

**usage [ext: DeriveAnyClass]**: derive a class with default implementations

```
{-# LANGUAGE DeriveAnyClass #-}

class MyClass a where                   -- class with
default
  myId :: a -> a
  myId = id

data MyType = A | B deriving (
  Show, Eq,                             -- derive
with stock strategy
  MyClass                               -- derive
with anyclass strategy
)

-- e.g.
myId A            -- A
myId B            -- B
```

**usage [ext:DerivingStrategies]**: select derivation strategy manually

```
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE StandaloneDeriving #-}

import GHC.Generics
import Data.Typeable
import Fmt

class MyClass a where
  myId :: a -> a
  myId = id

newtype Age = Age Int
  deriving stock (Eq, Show, Generic, Typeable)
  deriving newtype (Num, Build)
  deriving anyclass (MyClass)

deriving stock instance Eq a => Eq (Age a)
     -- strategy + standalone
deriving newtype instance Num a => Num (Age a)
deriving anyclass instance MyClass a => MyClass (Age a)

-- e.g.
Age 1 == Age 2          -- True    Eq i.e. a stock c
Age 1 + Age 2           -- Age 3
myId (Age 1)            -- Age 1
```

```
-- Monomorphic type
{-# LANGUAGE StandaloneDeriving #-}
deriving instance Eq MyType
```

```
data MyType = ... deriving (Eq)          -- same
```

```
-- Polymorphic type
deriving instance Eq a => Eq (MyType a)
deriving instance Eq MyType                -- ERROR if
MyType :: * -> *
```

```
data MyType a = ... deriving (Eq)        -- same
```

```
-- additional constraints
-- Only possible with standlone deriving
deriving instance Eq a => Eq (MyType [a])            --
deriving instance Eq a => Eq (MyType (Maybe a))

-- with strategies
{-# LANGUAGE DerivingStrategies #-}
deriving stock instance Eq a => Eq (MyType a)
deriving newtype instance Num a => Num (MyType a)
deriving anyclass instance SomeClass a =>
SomeClass (MyType a)

-- deriving via
{-# LANGUAGE DerivingVia #-}
newtype MyType m a = MyType (m a)
deriving via (m a) instance Show (m a) => Show (MyType m
a)      -- monomorphic   i.e.  Show :: Type ->
Constraint
deriving via m instance Functor m => Functor (MyType m)
         -- polymorphic   i.e.  Functor :: (Type -> Type)
-> Constraint
```